

Applications of AI Literacy to GenAI-Assisted Game Development

CSC 484/584 Game AI, North Carolina State University. 17 September 2026.

Slides	Download as PDF - 55 slides
Figures	37, each described in full

Everything on the slides is below, in order. The PDF is printed from this page, so the two say the same thing. The figure descriptions were written for this page; the deck did not carry any.

Slide 1

Applications of AI Literacy to GenAI-Assisted Game Development

- CSC 484/584 Game AI, guest lecture
- Sam Gilson, PhD student in Dr. Barnes' Game2Learn lab
- September 2026

Slide 2

Four parts

- Context, justification, and system architecture for easily deployable, browser based artifacts
- AI Literacy: Catching hallucinations, prompt engineering, collaborative prompting
- From design docs to a game prototype
- Ideas / Examples for project; multiplayer with no server, Q-learning for enemies

Slide 3

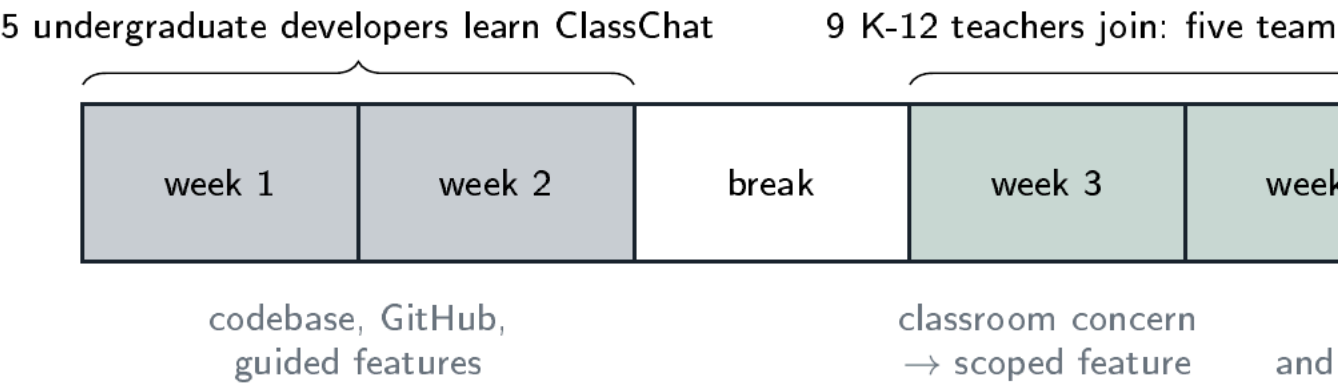
1. Why am I here talking to you?

- Motivations for Game Engine + AI Design

Slide 4

Exclaim Teacher Professional Development

- Six weeks around one classroom chatbot called ClassChat
- Developers spent two weeks learning it
- Then four weeks building tools for teachers, with teachers
- Five teams, five tools

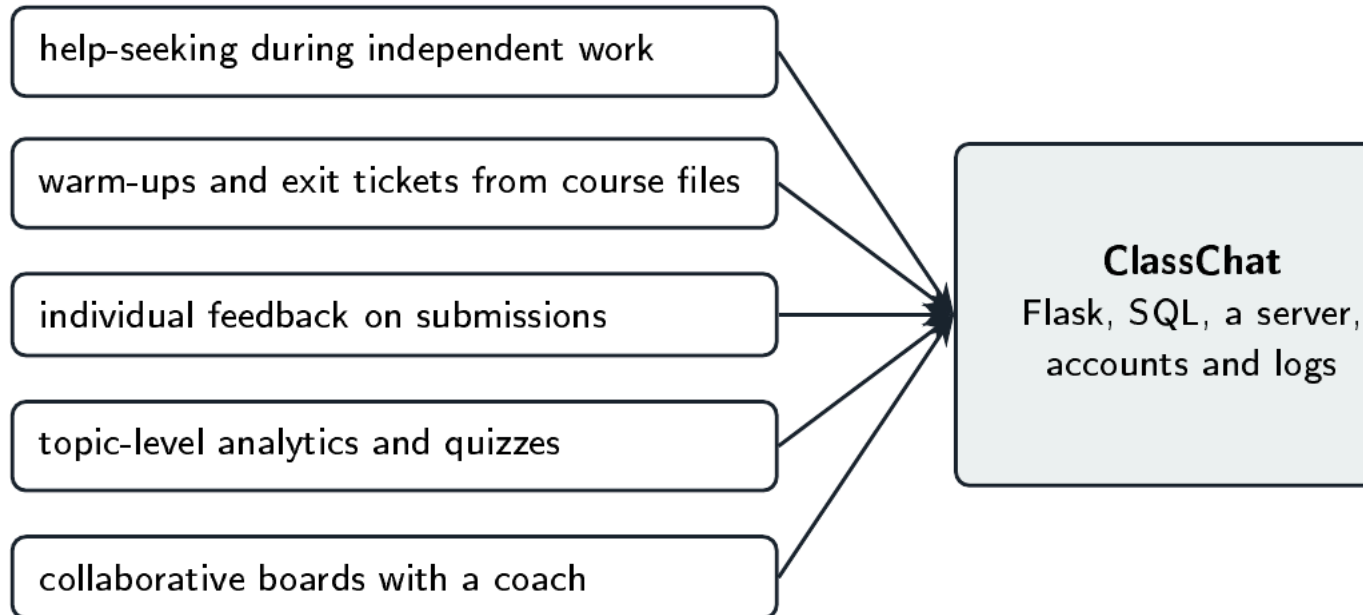


A six-week timeline drawn as a row of blocks. Weeks 1 and 2 are shaded under the heading '5 undergraduate developers learn ClassChat', covering codebase, GitHub and guided features. A break follows. Weeks 3 to 6 are shaded under '9 K-12 teachers join: five teams, one developer and one or two teachers each' -- week 3 takes a classroom concern to a scoped feature, weeks 4 and 5 are build with teacher and cross-team testing, and week 6 is revise, document, School AI Plan.

Slide 5

What teachers asked for

- Help when a student is stuck and won't ask
- Warm-ups and exit tickets from their own files
- Feedback on individual submissions
- Practice analytics and quizzes
- All of it went into the chatbot



Five labelled boxes with arrows converging on one box. The five are help-seeking during independent work; warm-ups and exit tickets from course files; individual feedback on submissions; topic-level analytics and quizzes; and collaborative boards with a coach. They all point at a box reading 'ClassChat: Flask, SQL, a server, accounts and logs', annotated as the one setting the developer knew and the teacher had trained on. A note beside it reads: so everything went in, whether or not a shared chatbot was the right home for it.

Slide 6

Findings relative to shared technical setting

developers said

- “always having to debug a lot” after AI wrote the foundation
- a change looked right and the app still failed: database or runtime state
- peers’ untested branches; long uploads timing out the server
- permissions and security roles left unclear, so features were held back

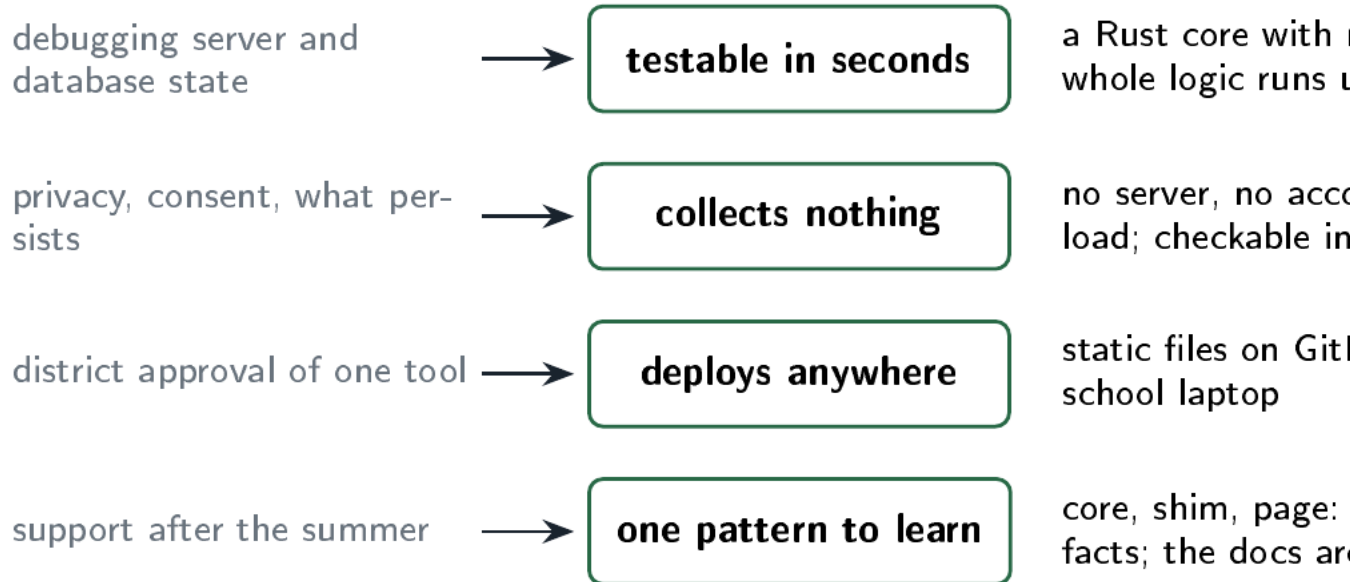
teachers said

- “debugged and fu a classroom
- district approval i single student AI
- privacy: what per dents consent to
- a fallback if the p after the summer

Two columns of quoted concerns. Under 'developers said': always having to debug a lot after AI wrote the foundation; a change looked right and the app still failed, in database or runtime state; peers' untested branches and long uploads timing out the server; permissions and security roles left unclear, so features were held back. Under 'teachers said': debugged and fully developed before it reaches a classroom; district approval required, with one district allowing a single student AI tool; privacy, meaning what persists, who sees it and what students consent to; a fallback if the platform is unavailable, and support after the summer.

Slide 7

Design Takeaways

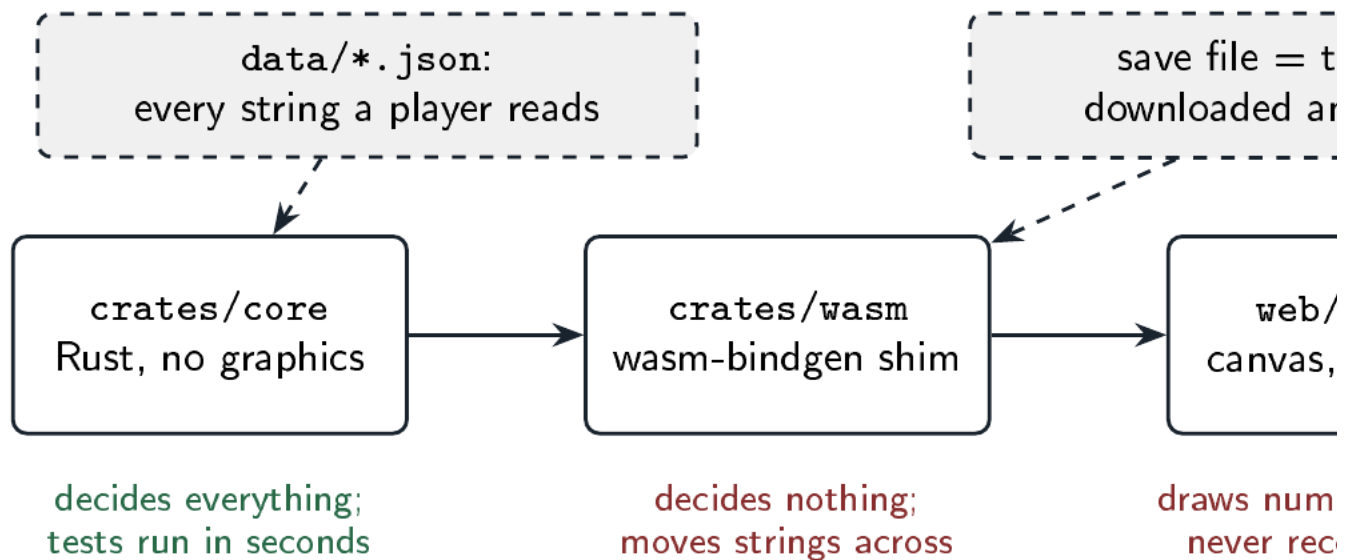


Four rows, each turning a constraint on the left into a design property on the right. Debugging server and database state becomes 'testable in seconds': a Rust core with no graphics or DOM, the whole logic running under cargo test. Privacy, consent and what persists becomes 'collects nothing': no server, no account, no network request after load, checkable in DevTools. District approval of one tool becomes 'deploys anywhere': static files on GitHub Pages, opened on any school laptop. Support after the summer becomes 'one pattern to learn': core, shim and page, learned once and used for many artifacts, with the docs as the handoff.

Slide 8

Rust + WASM + JS Architecture for deployable artifacts

- Rust core holds all the logic, no graphics
- A wasm shim to mediate transfer from rust core to graphical engine
- A plain JS page draws what core sends
- Static files on GitHub Pages
- Content is JSON; state is a save file

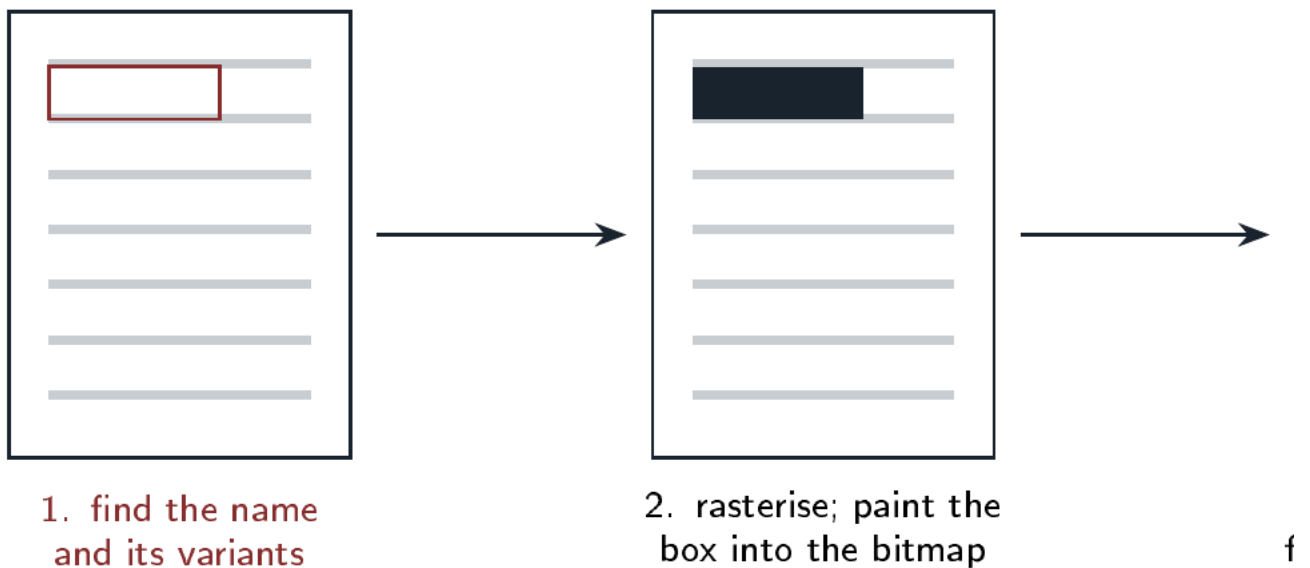


A left-to-right pipeline of four boxes. crates/core, Rust with no graphics, decides everything and its tests run in seconds. crates/wasm, a wasm-bindgen shim, decides nothing and moves strings across. web/app.js, canvas and vanilla JS, draws the numbers core sent and never recomputes one. GitHub Pages static files: no server, no account, no network after load. Two notes point in: data/*.json holds every string a player reads, and the save file is the database, downloaded and re-uploaded.

Slide 9

Proof of Concept 1

- PDF redactor



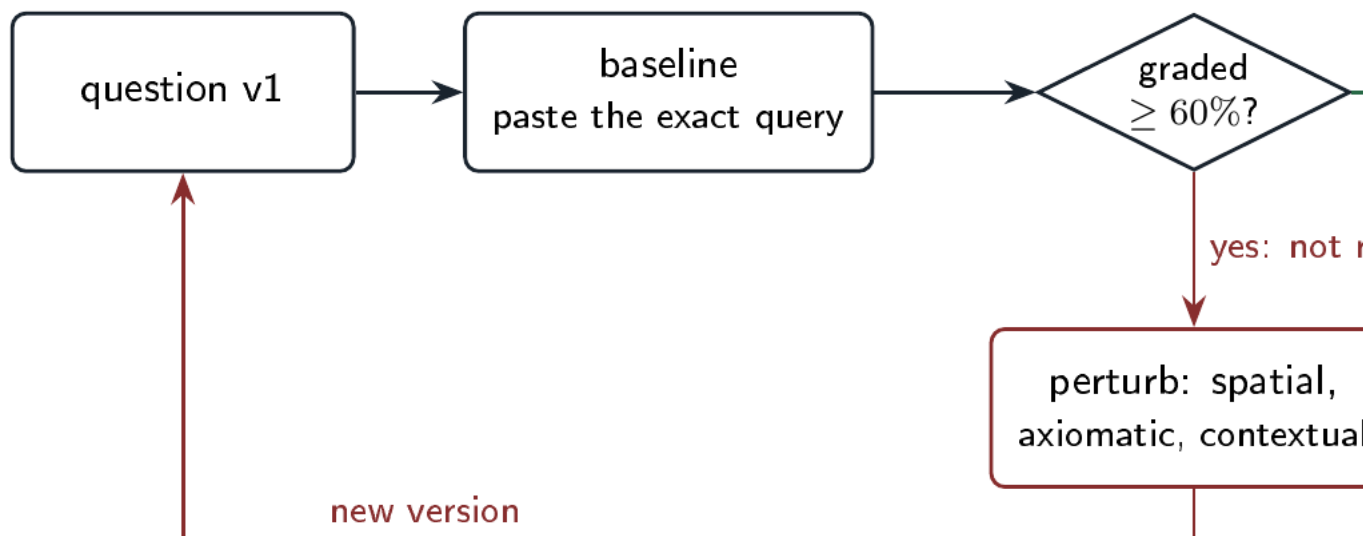
Three document outlines in sequence. In the first, a name on the page is ringed in red: find the name and its variants. In the second that area is a solid black block: rasterise, and paint the box into the bitmap. In the third the solid block remains: write a new PDF from the bitmaps only.

Slide 10

Proof of Concept 2

- Perturbation Workbench
- Goes with our AIED 2026 paper
- You paste the query into a fresh chatbot
- Grade it against a rubric you fixed first
- Too easy? Perturb it and try again

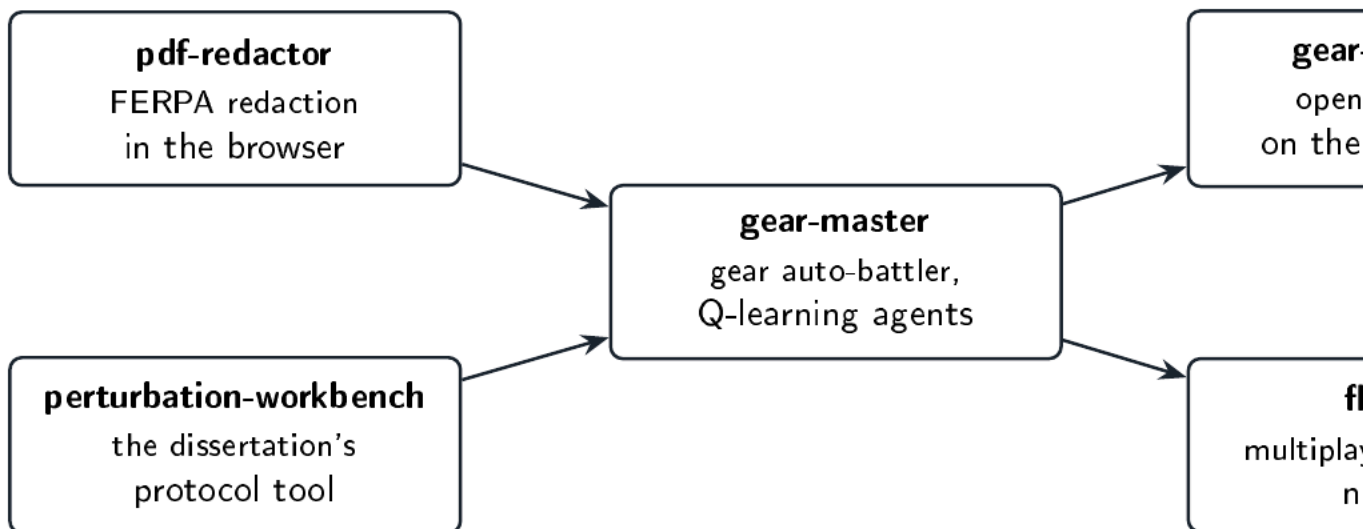
- Three misses in a row, and it's resistant



A flowchart. 'Question v1' leads to a baseline step, pasting the exact query, then to a decision diamond asking whether it graded at or above 60 percent. The 'no, three times' branch leads to a box marked resistant, three attempts below. The 'yes: not resistant' branch leads to a perturb step offering spatial, axiomatic and contextual changes, which loops back to the question as a new version.

Slide 11

Proof of Concepts worked, so I made some games



Five repository boxes with arrows. pdf-redactor, FERPA redaction in the browser, and perturbation-workbench, the dissertation's protocol tool, both feed into gear-master, a gear auto-battler with Q-learning agents. gear-master in turn feeds gear-master-2d, an open-world RPG on the same engine, and floodline, a multiplayer city builder with no server.

Slide 12

2. AI Literacy

- How a chatbot writes a proof

Slide 13

One token at a time

- Each token is a draw from a distribution

- Conditioned on the internet
- Pick the likely one, append, repeat
- Nothing in that loop checks first principles and logical validity
- It sounds like a textbook because it read textbooks

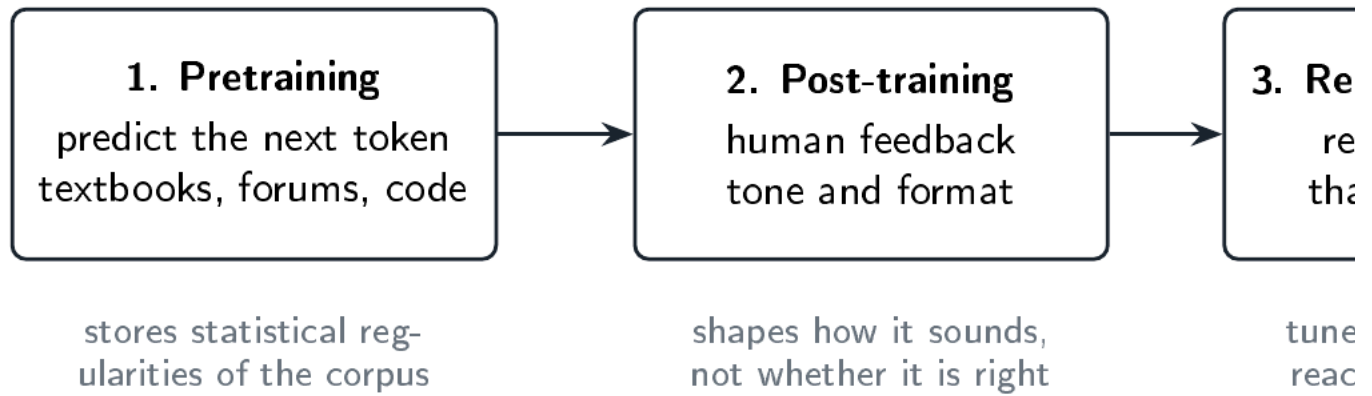


No step in that loop checks whether any of it is valid.

Four sampling steps, labelled T_0 to T_3 , building the sentence 'Prove that for all n at least 1, the integer n is'. At each step the next word is drawn from a short list of candidates with probabilities: 'the' at 0.42 against we 0.31, by 0.18 and sum 0.05; then 'integer' at 0.95; then ' n ' at 0.71; then 'is' at 0.46. A final row, T_n , reads: a finished proof, several hundred more draws, each conditioned on everything above it. Underneath, in red: no step in that loop checks whether any of it is valid.

Three stages of training

- Pretraining: predict the next token
- Post-training: tone and format
- RL: reward for answers that can be checked
- None of them stores a rule of inference



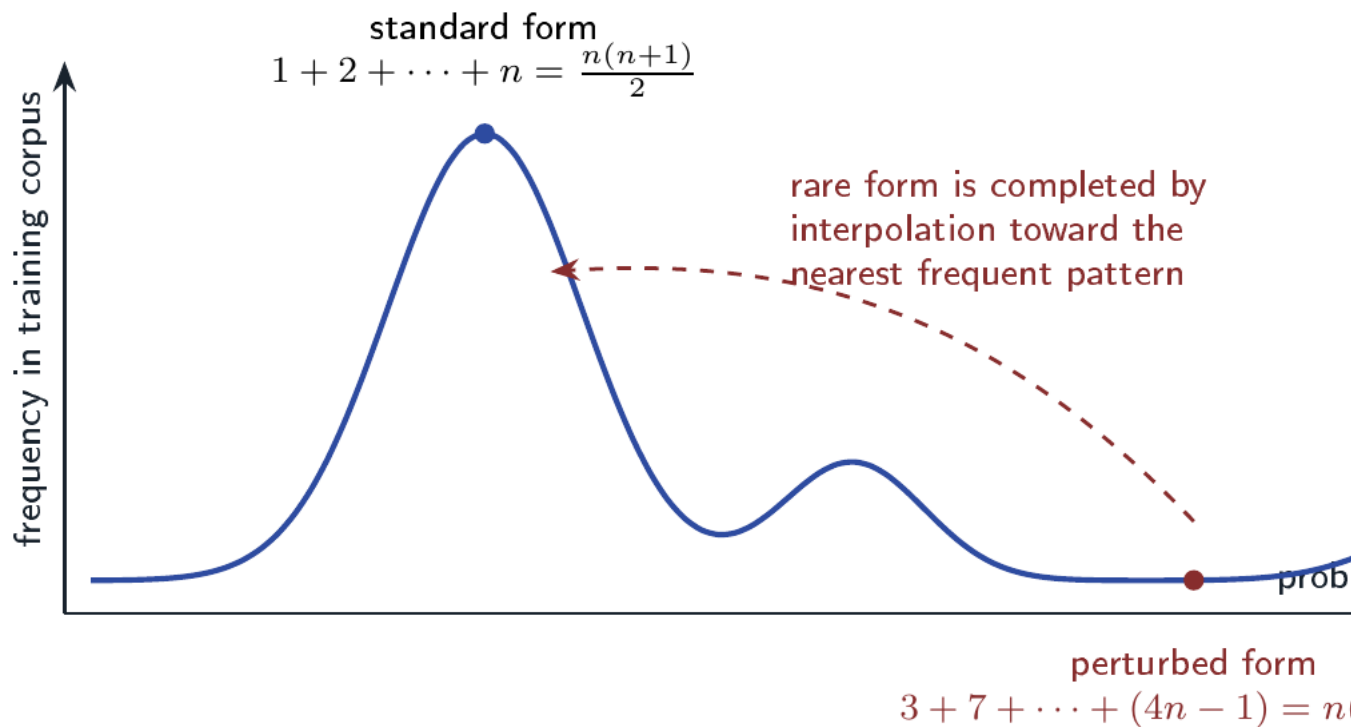
No stage stores rules of inference.

Three boxes in sequence. The first is labelled pretraining, "predict the next token", from textbooks, forums and code, with a caption reading "stores statistical regularities of the corpus". Post-training uses human feedback for tone and format, and shapes how it sounds rather than whether it is right. Reinforcement learning rewards answers that can be checked, and tunes toward chains that reach checkable answers. Underneath, in red: no stage stores rules of inference.

Slide 15

Unfamiliar questions more likely to hallucinate

- Reliability tracks how common the problem is
- $1+2+\dots+n$: thousands of times in the corpus
- $3+7+11+\dots$: same structure, almost never
- The rare one gets filled in from the common one
- No harder for a person



A curve of frequency in the training corpus against problem formulations. A tall peak is labelled with the standard form, $1 + 2 + \dots + n = n(n+1)/2$. Far out along the flat tail sits a marked point labelled with the perturbed form, $3 + 7 + \dots + (4n - 1) = n(2n + 1)$. A dashed arrow runs from the rare point back up toward the peak, labelled: rare form is completed by interpolation toward the nearest frequent pattern.

Slide 16

Three ways chatbots mess up when solving proofs

Axiom reversion

swaps in the textbook theorem the problem was perturbed from

indicator: closed form matches the text-book, not the problem

Invalid syntactic shortcut

manipulates the form as if a rule applied; well-formed but false

indicator: the step fails when checked symbolically

Spatial or counting miscount

counts placements that break the constraint, or double counts

indicator: the case breakdown does not sum to the total

Three cards, one per failure mode. Axiom reversion swaps in the textbook theorem the problem was perturbed from; its indicator is that the closed form matches the textbook rather than the problem. Invalid syntactic shortcut manipulates the form as if a rule applied, well-formed but false; its indicator is that the step fails when checked symbolically. Spatial or counting miscount counts placements that break the constraint, or double counts; its indicator is that the case breakdown does not sum to the total.

Slide 17

Procedure A — The hallucination autopsy procedure

Procedure A — The hallucination autopsy procedure		
Label	What the step is for	What you do
A0	Calibrate your reliance	Before reading the response, note how much you expect to trust it for this problem type, and why. Revisit after A6.
A1	Restate the claim as given	Write down exactly what is to be proven, including its right-hand side.
A2	Extract the rule the chatbot actually used	Find the closed form it substituted during the inductive step.
A3	Locate the first divergence	Mark the exact line where the stated hypothesis and the substituted rule part ways.
A4	Test a small case	Evaluate both sides at a small n. If the numbers agree, verify one derivation step anyway.
A5	Classify the failure mode	Axiom reversion, invalid syntactic shortcut, or spatial / counting miscount.
A6	Explain why the failure was likely	Connect the error to how the model generates text.

Slide 18

Catching a hallucination

- A1 A2 A3 A4 A5 A6
- Claim: $5+9+13+\dots+(4n+1) = n(2n+3)$
- The chatbot slipped in $k(k+1)/2$
- The hypothesis says $k(2k+3)$
- Try $n=2$: 14 on one side, 3 on the other
- That's the textbook coming back

A1 claim: $5 + 9 + 13 + \dots + (4n + 1) = n(2n + 3)$

$$\begin{aligned} & 5 + 9 + \dots + (4k + 1) + (4(k + 1) + 1) \\ &= \frac{k(k + 1)}{2} + (4k + 5) \quad \text{“substituting the standard Gauss sum”} \\ &= (k + 1)(2(k + 1) + 3) \quad \blacksquare \end{aligned}$$

A4 $n = 2$: the claim gives $5 + 9 = 14$; the chatbot’s form gives $\frac{2 \cdot 3}{2} = 3$. wrong. Had they agreed: check one step anyway.

A5 axiom reversion

A6

the Gauss sum is everywhere in the corpus; the term of the sum and kept the familiar closed form

A worked example annotated with the procedure’s step labels. A1 gives the claim, $5 + 9 + 13 + \dots + (4n + 1) = n(2n + 3)$. A boxed derivation substitutes $k(k+1)/2$, described in the chatbot’s words as substituting the standard Gauss sum, with that substitution ringed in red. A2 names the rule actually used, $k(k+1)/2$. A3 marks the first divergence: the hypothesis says $k(2k + 3)$. A4 tests $n = 2$, where the claim gives $5 + 9 = 14$ and the chatbot’s form gives 3, so they disagree and the proof is wrong; had they agreed, check one step anyway. A5 classifies it as axiom reversion. A6 explains it: the Gauss sum is everywhere in the corpus, so the model matched the visual pattern of the sum and kept the familiar closed form.

Slide 19

Procedure B — Structure of a strong prompt

Procedure B — Structure of a strong prompt

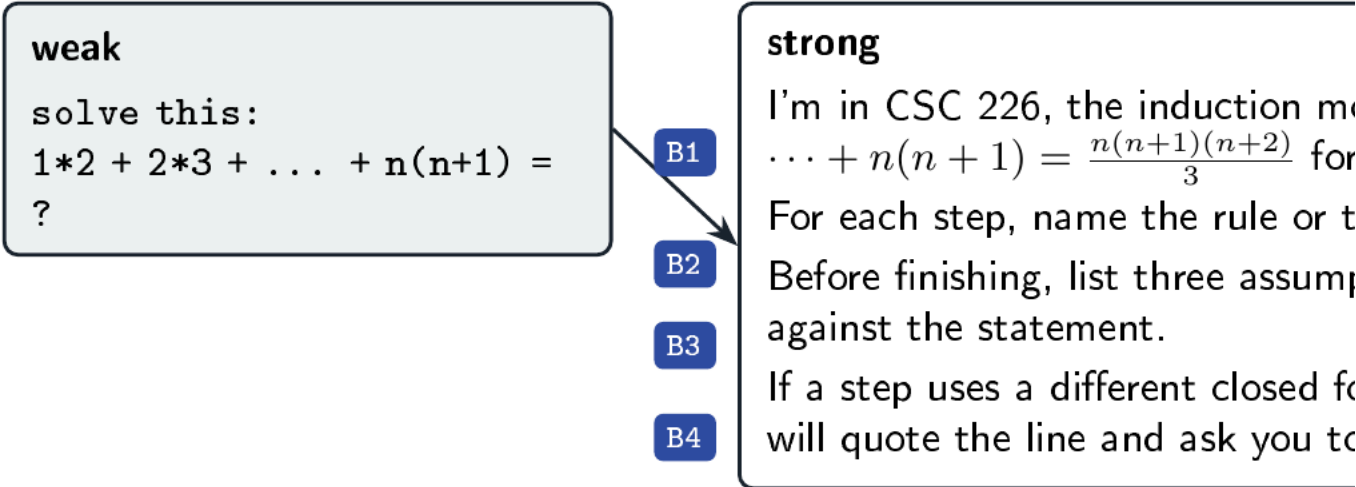
Label	What the step is for	What you do
B1	Establish context	Tell the chatbot which subject, question type, and goal.
B2	Require step-by-step justification	Require each step to name the rule or prior line it uses.
B3	Build in self-questioning	Before the chatbot finishes, have it list its assumptions and check them.
B4	Plan the corrective dialogue	Know in advance what you will say if you spot an error.

Same label, same words, every time it appears: in this box, in a worked example, and in your own task.

Slide 20

Fixing a weak prompt

- B1 B2 B3 B4
- Weak: “solve this: $1 \cdot 2 + 2 \cdot 3 + \dots = ?$ ”
- Say which course, which module, which claim
- Ask for the rule at every step
- Ask it to list and check its assumptions
- Write the correction before you need it



Two prompts side by side. The weak one reads in full: solve this: $1*2 + 2*3 + \dots + n(n+1) = ?$ The strong one is annotated with step labels. B1 establishes context: I'm in CSC 226, the induction module, prove $1 \times 2 + 2 \times 3 + \dots + n(n+1) = \frac{n(n+1)(n+2)}{3}$ for n at least 1. B2: for each step, name the rule or the earlier line it uses. B3: before finishing, list three assumptions and check each against the statement. B4: if a step uses a different closed form than my hypothesis, I will quote the line and ask you to redo it.

Slide 21

Procedure C — Scaffolding strong prompts to provide critical feedback

Procedure C — Scaffolding strong prompts to provide critical feedback		
Label	What the step is for	What you do
C1	Frame the problem and set the rules	Give context, state the problem exactly, forbid the final answer.
C2	Elicit one step with its rule named	Ask for the next step and the counting rule it uses.
C3	Interrogate the claim	Request a justification or issue a correction; never accept a step silently.
C4	Verify against hand-computed cases	Small cases computed by hand decide who is right.
C5	Consolidate in your own words	The write-up is yours, not the chatbot's.
C6	Question the system	Ask who built it, on whose data, and why you trust its answer.

Slide 22

If you could make a game right now, what would it be?

Slide 23

Do you think you could make a prototype of your game, and implement AI control of entities for the class project?

Slide 24

Would you incorporate generative AI into your game?

Slide 25

Would you incorporate generative AI into your game?

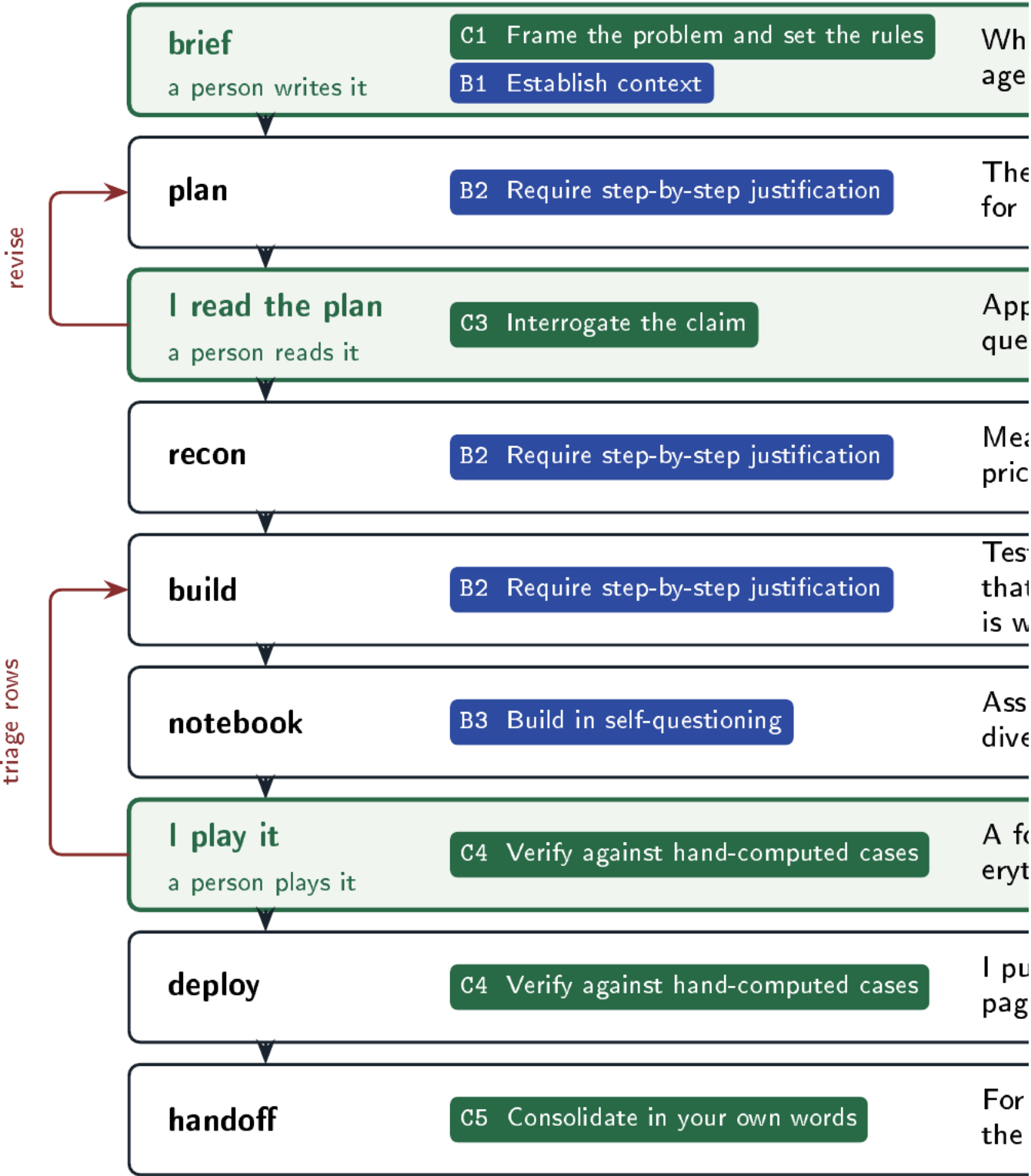
- If you would, what would you use it for?
- If you would not, explain why?

Slide 26

3. Generative AI for Game Prototyping

- Using AI literacy to optimize agentic game engine development workflows

GenAI-Assisted Prototyping Procedure



A nine-stage vertical pipeline, each stage tagged with the procedure steps it serves. Brief, written by a person, tagged C1 and B1: what we are making, what done means, and what the agent may not decide on its own. Plan, B2: the agent fills the brief in, every decision and the reason for it, before any code. 'I read the plan', read by a person, C3: approve it or send it back, and a reference it cannot find is a question for me rather than a guess; a revise arrow loops back to plan. Recon, B2: measure against the running build, counts, brackets and prices, and say in the commit what you found. Build, B2: tests and content lints in the same change, and a number that disagrees with the plan must name which of the two is wrong. Notebook, B3: assumptions listed and checked while the work happens, as a divergence, a finding, or a decision handed back. 'I play it', played by a person, C4: a fortnight of real play against what the plan claimed, with everything wrong becoming a triage

row, looping back to build. Deploy, C4: I push, and drive.py then walks every check against the live page in three engines. Handoff, C5: for whoever is next, the rules that are load-bearing and the mistakes that cost a day.

Slide 28

Project Planning Artifacts

PLANNING-BRIEF.md	you	what are we making, and what does done mean	C1
CLAUDE.md	both	the load-bearing rules, and the mistakes that cost a day	C1 A6
TONE.md	you	ten checkable rules for every string a player reads	B1
PLAN.md, PLAN-Mn.md	agent	the decisions, with a paragraph of why for each	B2
PROMPT-Mn.md	you	reading order, your ask verbatim, recon first	B1 B2
SECOND-ORDER-Mn.md	agent	every assumption checked while the work is done	B3 C3
PLAYTEST, TRIAGE	you	what a fortnight of play reported, sorted into rows	C4 B4
HANDOFF.md	agent	what the next reader needs before touching anything	C5

A table of planning documents, who writes each, and the procedure steps behind them. PLANNING-BRIEF.md, written by you: what are we making, and what does done mean; C1. CLAUDE.md, both: the load-bearing rules, and the mistakes that cost a day; C1 and A6. TONE.md, you: ten checkable rules for every string a player reads; B1. PLAN.md and PLAN-Mn.md, the agent: the decisions, with a paragraph of why for each; B2. PROMPT-Mn.md, you: reading order, your ask verbatim, recon first; B1 and B2. SECOND-ORDER-Mn.md, the agent: every assumption checked while the work is done; B3 and C3. PLAYTEST and TRIAGE, you: what a fortnight of play reported, sorted into rows; C4 and B4. HANDOFF.md, the agent: what the next reader needs before touching anything; C5.

Slide 29

Unit and System Testing

- Unit tests on the core, in seconds
- Lints on the content, not just the code
- Save round-trip on every commit
- A browser gate, three engines
- Then the same gate against the live page

core unit tests

A4 Test a small case

1,037 tests in 90 binaries: combat, saves, maps, economy

content lints

B3 Build in self-questioning

every flag an event sets is read somewhere; every refusal names its reason

save round-trip

C1 Frame the problem and set the rules

a new field is a compile error until the save carries it

browser gate

C4 Verify against hand-computed cases

every check in three engines, then again on the live page

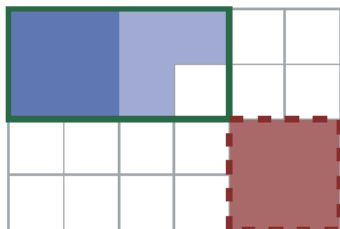
A check nobody has seen fail is not a check: three shipped comparing zero with zero. Break a new one and watch it go red before you keep it.

Four stacked bars, one per layer of testing. Core unit tests, tagged A4: 1,037 tests in 90 binaries covering combat, saves, maps and economy, on every change, in seconds. Content lints, B3: every flag an event sets is read somewhere, and every refusal names its reason, on every change. Save round-trip, C1: a new field is a compile error until the save carries it, on every commit. Browser gate, C4: every check in three engines, then again on the live page, before and after deploy. Underneath: a check nobody has seen fail is not a check -- three shipped comparing zero with zero. Break a new one and watch it go red before you keep it.

Slide 30

Prototype 1: Gear Master

- Inspired by backpack battles
- Five slots, each one a grid
- Drag parts in; touching parts become an item
- Finished items fire on their own cooldowns
- Press fight, then watch



core + touching pieces that match a recipe = an **item**. Gold outline: finished.

dashed: pieces placed, recipe not met. They still add base stats; bonuses wait.

BEGIN

each finished
cooldown
creature's be
until it e

the weapon slot (one of five)

A five-by-five grid holding pieces. A green outline rings a finished item made of a dark core square and a lighter square touching it; a red dashed square sits apart from them. Notes explain that a core plus touching pieces matching a recipe makes an item, with a gold outline meaning finished, while dashed means pieces placed without the recipe met, so they still add base stats and the bonuses wait. The grid is captioned 'the weapon slot (one of five)'. A panel reads: BEGIN FIGHT -- each finished item fires on its own cooldown, your board above and the creature's below, and nobody presses anything until it ends.

Slide 31

Prototype 2: Gear Master 2D

- Same core engine as Gear Master!



walk a tile a press; terrain sets the encounter rate

creatures

drawn by rating
the board auto-

levels and classes

a level adds a r
level; a class at

towns and errands

shops sell comp
21 maps now

dungeons

several floors; e
or gates before

save file

download any t
cluding the RN

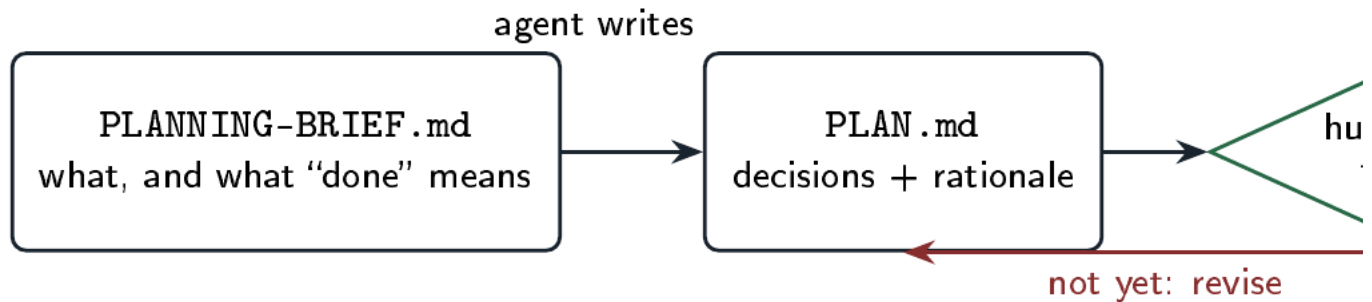
A tiled overworld map with a dark dot labelled 'you' and a red dot labelled 'town', captioned: walk a tile a press, terrain sets the encounter rate. Beside it, five notes. Creatures are drawn by rating from the region's pool, and a fight is the board auto-battle. Levels and classes: a level adds a row to a slot grid, one skill point a level, a class at level 5. Towns and errands: shops sell components, errands are the quest log, 21 maps now. Dungeons have several floors, each a puzzle of stakes, levers or gates before the boss. The save file downloads any time, and loading restores everything including the RNG.

Slide 32

Design Insight 1No code until I've read the plan

- C1 Frame the problem and set the rules

C1 “Execute Part A to produce PLAN.md. Stop and show me PLAN.n



A flow from PLANNING-BRIEF.md, what and what done means, to PLAN.md, decisions plus rationale, written by the agent, then to a diamond where a human reads the plan. The approved branch leads to milestone 0; the other branch, 'not yet: revise', loops back to PLAN.md. Above it, tagged C1, the instruction quoted in full: Execute Part A to produce PLAN.md. Stop and show me PLAN.md before executing any milestone.

Slide 33

Design Insight 2

- How to prompt an agentic chatbot with design docs
- B1 Establish context
- B2 Require step-by-step justification

B1	reading order	Read HANDOFF.md, then CLAUDE.md R then PLAN-M16.md
B1	the ask, verbatim	“add a dungeon in the wextreen sands . . . v its gear slots”
B2	recon before content	measure the boss by bracket against the sa boss
B2	findings in the commit	put what the recon found in the commit m which is wrong and why
B3	keep the notebook	SECOND-ORDER-Mn.md: write a row the moi effect, not saved up for the end
C1	deploy points	after M16.3 and after M16.5; a human dep page

Six rows pairing a part of the prompt with what it asked for. B1, reading order: read HANDOFF.md, then CLAUDE.md Rules, then these four sections, then PLAN-M16.md. B1, the ask verbatim: add a dungeon in the wextreen sands, with my current run's loadout as its gear slots. B2, recon before content: measure the boss by bracket against the save, never by adding to the last boss. B2, findings in the commit: put what the recon found in the commit message, and if a count disagrees, say which is wrong and why. B3, keep the notebook: write a row in SECOND-ORDER-Mn.md the moment you notice a second-order effect, not saved up for the end. C1, deploy points: after M16.3 and after M16.5, with a human deploying and then checking the live page.

Slide 34

Design Insight 3

- Managing Second Order Effects

- B3 Build in self-questioning
- C3 Interrogate the claim
- Second Order Effect notebook explicitly asked for in prompt
- Written while the work is happening
- One row per assumption
- Divergence, finding, or worklist?
- Worklist is brought to the human for guidance on; typically large reaching changes
- Example: Seven rows in one block turned up real bugs

#	row	kind
4	the plan's curse-cap numbers do not reproduce, in either direction	div
9	solvable__blind is 36 and the plan asks for 9: pulls vs card reads	div
14	is rarity a creature's measure or a player's? three answers, all content	wo
17	a data/ change costs three minutes of relinking; measured, not fixed	wo
25	enemies.json went lossy the moment a creature carried an ench	fin

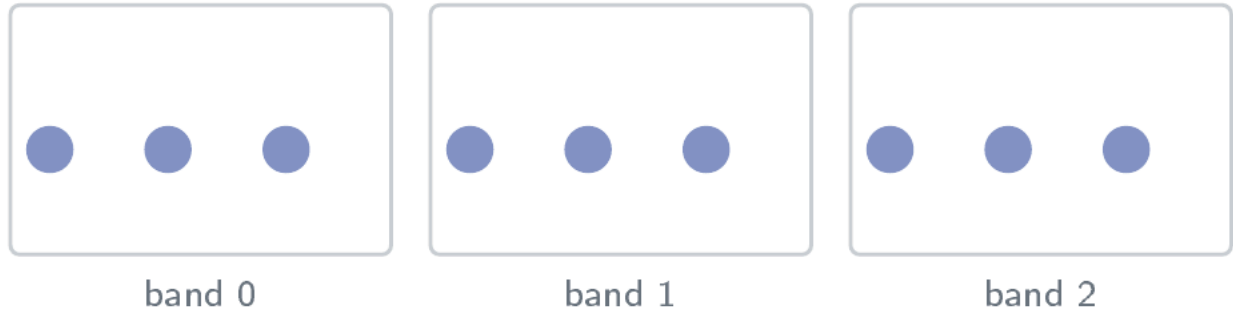
A table of notebook rows, each with a number, the row, its kind and its status. Row 4: the plan's curse-cap numbers do not reproduce, in either direction; a divergence; done. Row 9: solvable__blind is 36 and the plan asks for 9, pulls versus card reads; a divergence; done. Row 14: is rarity a creature's measure or a player's, three answers and all content; a worklist row; the human's. Row 17: a data/ change costs three minutes of relinking, measured rather than fixed; a worklist row; done. Row 25: enemies.json went lossy the moment a creature carried an ench; a finding; done.

Slide 35

Bug caught by Second Order Notebook

- A2 Extract the rule the chatbot actually used
- A3 Locate the first divergence
- A5 Classify the failure mode
- A puzzle floor
- nine stakes in three bands that must be pulled in order to open the door to the next floor
- Divergence: The plan used 9 stakes; the test used all 36 stakes
- Agent added the divergence to the second order notebook
- Human Root Cause Analysis:
- Test was written to try every possibility and assumed every possibility should pass
- Reality: only 9 of them should pass

nine stakes, three gated bands, one right stake per band



- A2** rule used: count pulls **A3** divergence: the unit changed mid-plan
- A5** counting miscount **fix** assert both numbers, each in its own unit, come back unnoticed

Three bands, each holding three dots, labelled band 0, band 1 and band 2, under the heading 'nine stakes, three gated bands, one right stake per band'. Two panels contrast the numbers. 9 is the plan, counting pulls, one per stake. 36 is the test, counting card reads, where the adversary opens each band early and leaves two wrong neighbours live. Tags: A2, the rule used, count pulls; A3, the divergence, the unit changed mid-plan; A5, counting miscount; and a fix -- assert both numbers, each in its own unit, so the disagreement cannot come back unnoticed.

Slide 36

Design Insight 4

- Avoid multiple sources of truth
- C4 Verify against hand-computed cases
- We'd been saying 1,137 tests
- Three ways of parsing gave three numbers
- cargo mixes its output streams
- Actual source of truth: Check the binaries directly
- 1,037, on any machine

```
test result: ok. 28 passtest every_place_on
_a_shot_map_is_reachable_by_shots ... ok
test result: ok. 14 passed; 0 failed
test resutest result: ok. 9 passed
```

packaging/count-tests.sh
builds the binaries
--message-format=json
--list, counts lines

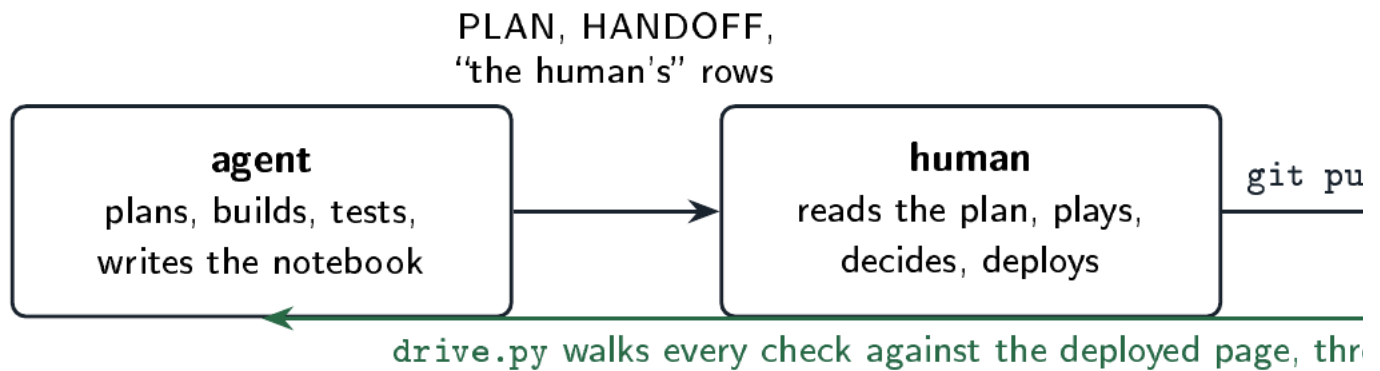
cargo interleaves test binaries on one stream. Summing the fourth field gave **978, 1,025 and 1,053** from the same run. 1,137 was quoted three times.

The same command gives the same number on any machine

A dark terminal block showing cargo output running together on one stream, with fragments such as 'test result: ok. 28 passtest every_place_on_a_shot_map_is_reachable_by_shots ... ok' and 'test resutest result: ok. 9 passed'. The caption explains that cargo interleaves test binaries on one stream, so summing the fourth field gave 978, 1,025 and 1,053 from the same run, while 1,137 was quoted three times. An arrow points to packaging/count-tests.sh, which builds the binaries, finds them from --message-format=json, runs each with --list and counts lines: the same command gives the same number on any machine, 1,037 tests in 90 binaries.

Design Insight 5

- Who does what during prototyping
- C1 Frame the problem and set the rules
- C5 Consolidate in your own words
- The agent plans, builds, tests, writes it up
- I read, play, decide, deploy
- It never pushes on its own
- After a deploy, the testing gates walk the live page
- Handoff docs are written around at ~700k tokens used in current context
- Around there, model performance starts to degrade
- Handoff doc is to inform context-cleared model how to pick up the work



C1

the agent does not push on its own judgement

C5

HANDOFF.md is
human's terms

Three boxes left to right. The agent plans, builds, tests and writes the notebook. The human reads the plan, plays, decides and deploys, and receives PLAN, HANDOFF and the rows marked 'the human's'. A git push arrow leads to the live page on GitHub Pages, the only thing players see. A return arrow underneath reads: drive.py walks every check against the deployed page, three browsers. Tagged C1, the agent does not push on its own judgement, and C5, HANDOFF.md is written for the next reader in the human's terms.

What sorts of benefits and drawbacks do you think arise from Generative AI use for Game Prototyping?

4. Game AI

- Concepts and stuff you can borrow for the project

Floodline

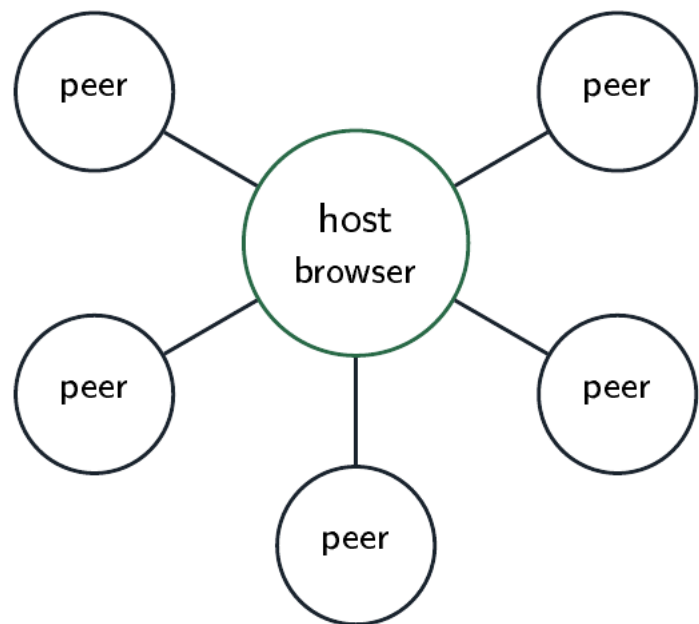
- Multiplayer without a server
- Up to six players share one map

- The host's browser is the hub
- A public relay introduces peers, once

star: only commands travel, a few kB/s for six brow

GitHub Pages serves the files.
After that, nothing runs anywhere.

The one thing that can cost money: a TURN relay, for the rare strict NAT. One edit, no rebuild.

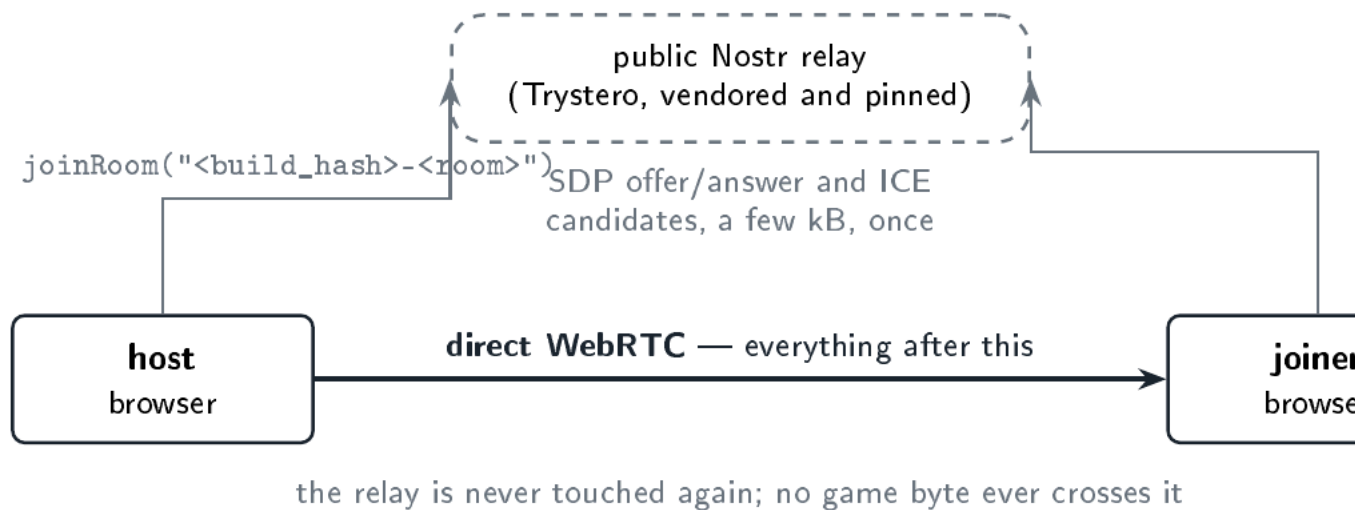


Five peer circles arranged around a central host browser, each joined only to the host, under the heading 'star: only commands travel, a few kB/s for six browsers'. Notes explain that GitHub Pages serves the files and after that nothing runs anywhere, and that the one thing which can cost money is a TURN relay for the rare strict NAT, one edit and no rebuild. Two panels: introduction happens once through a public relay, Nostr by default, never touched again; or a pasted code of about 300 characters each way over any chat, with no third party at all.

Slide 41

How two browsers find each other

- Two browsers cannot dial each other by address, so before a direct link exists they have to exchange an SDP offer, an answer and a handful of ICE candidates through some third channel. That channel is the only part of this that anyone else operates, and the game stops using it the moment the link is up.



fallback: paste a code

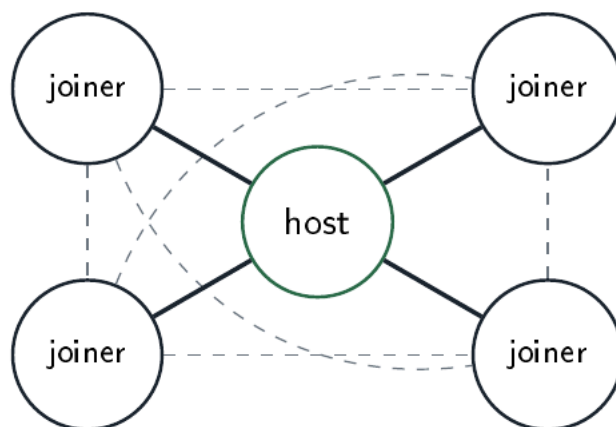
host gathers ICE to completion (no trickle), strips the lines the far end can infer, deflates, base64: ≈ 300 characters over any chat. One round trip, no third party, same two channels at the end.

Host and joiner browsers each reach up to a public Nostr relay, labelled Trystero, vendored and pinned, through `joinRoom` of build hash and room, exchanging an SDP offer and answer and ICE candidates, a few kB, once. A heavy arrow runs directly between the two browsers, labelled 'direct WebRTC -- everything after this', with the note that the relay is never touched again and no game byte ever crosses it. A fallback panel: the host gathers ICE to completion with no trickle, strips the lines the far end can infer, deflates and base64-encodes to about 300 characters over any chat -- one round trip, no third party, the same two channels at the end.

Slide 42

Trystero Room as a mesh network

- Every peer in a Trystero room meets every other peer. The lockstep wants joiners talking only to the host.



dashed: the mesh Trystero gives you
solid: the star the game wants

the first byte on the reliable ch

0x48 **H** from a host, 0x4A **J** from moment the channel opens. A host reports a peer. A joiner that reads host. A joiner that reads J has met it closes its two channels and tells

"first peer is the host" is a guess — meet before either meets the host. connection buys an invariant instead

channel 40, ordered — turns and **channel 41**, `maxRetransmits:0` — chat.

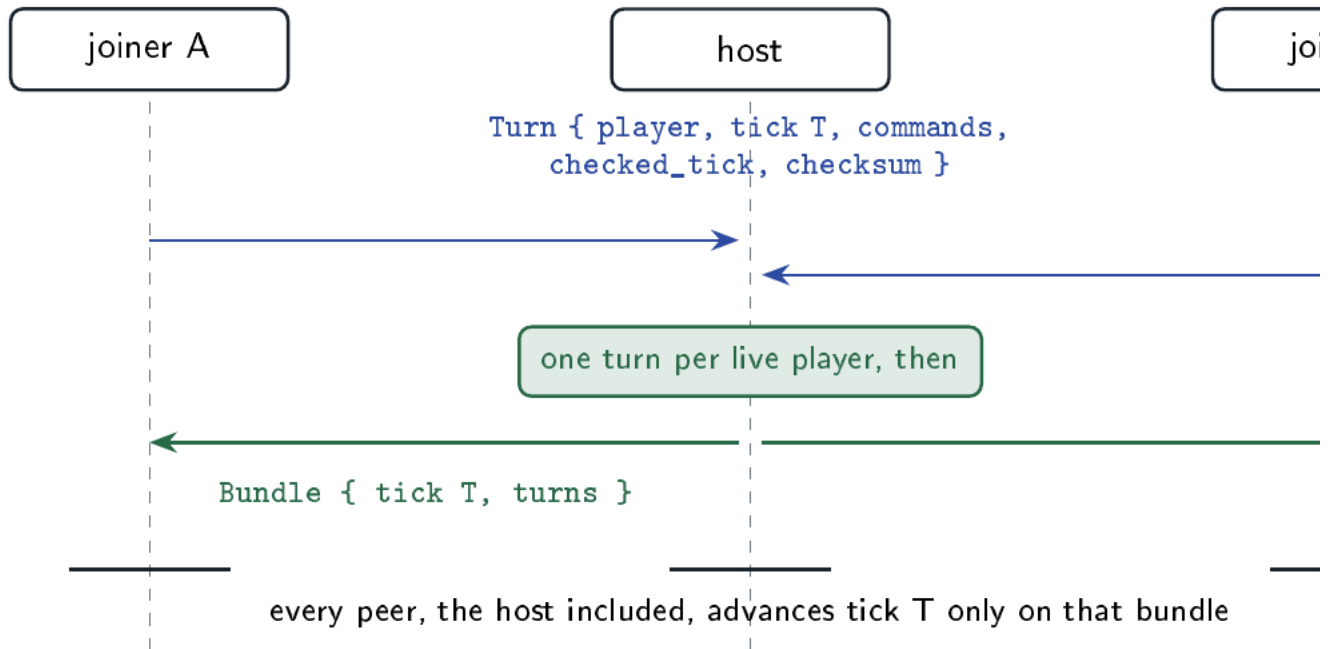
Ids 40 and 41, not 0 and 1: Trystero channel allocates from 0 upwards.

Four joiner circles around a host. Dashed lines connect every peer to every other, labelled 'the mesh Trystero gives you'; solid lines connect each joiner to the host alone, labelled 'the star the game wants'. Beside it, the first byte on the reliable channel is 0x48 H from a host and 0x4A J from a joiner, sent the moment the channel opens: a host that reads J reports a peer, a joiner that reads H has found the host, and a joiner that reads J has met another joiner, so it closes its two channels and tells Rust nothing. A note adds that 'first peer is the host' is a guess, because two joiners can meet before either meets the host, so one byte per connection buys an invariant instead. Channel 40 is ordered, for turns and bundles; channel 41 uses maxRetransmits 0, for cursors at 20 Hz and chat; the ids are 40 and 41 rather than 0 and 1 because Trystero's own in-band channel allocates from 0 upwards.

Slide 43

One tick, end to end

- Every peer sends the host one turn per tick and waits. The host collects a turn from each live player, bundles them, and sends the bundle back to everyone including itself. That bundle is the only thing any peer advances on.



A command issued at tick T is scheduled for $T + 3$ — 150 ms at twenty ticks a second. Long enough for a round trip, short enough that a click still feels like it did something.

Nobody simulates ahead and nobody rolls back; the host runs the same sim as everyone else, a relay with a clock rather than an authority, so a disagreement is reported rather than won.

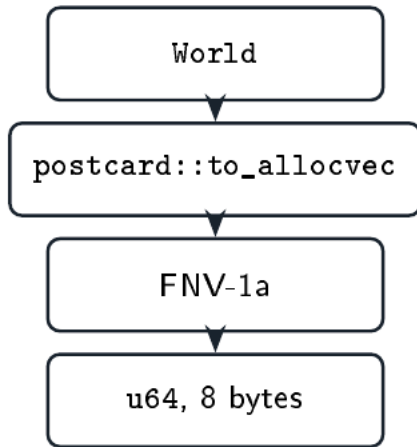
A sequence diagram with joiner A, the host, and joiner B. Each joiner sends the host a Turn carrying player, tick T, commands, checked_tick and checksum. The host collects one turn per live player, then sends both peers a Bundle of tick T and the turns. Every peer, the host included, advances tick T only on that bundle. Notes: a command issued at tick T is scheduled for $T + 3$, which is 150 ms at twenty ticks a second, long enough for a round trip and short enough that a click still feels like it did something. Nobody simulates ahead and nobody rolls back; the host runs the same sim as everyone else, a relay with a clock rather than an authority, so a disagreement is reported rather than won.

Slide 44

The checksum is the whole world, cheaply

- Every turn carries a 64-bit FNV-1a hash of the postcard encoding of the entire World. Hashing the encoding rather than a hand-written list of fields means a new field is covered the day it is added — and it is the same encoding a late joiner receives.

the checksum



Anything in the world is in the checksum *by construction* — no list of fields to forget to update.

It is the same encoding a late joiner receives, so if the checksums agree the snapshot will too.

Tagged with `checked_tick`: a turn for `T` is sent three ticks early, so the sender cannot yet know `T-1`.

per tick, on the wire

a player id, that tick's commands (usually none), and eight bytes of checksum — the host's bundle of the same.

never on the wire

world state, positions, paths -- flow fields are two vectors of sixteen thousand cells each, so Nav lives outside World: rebuilt identically on every peer, in nobody's snapshot and nobody's checksum.

once, on late join

Welcome { seed, tick and snapshot, 50 to 150 kB of postcard at five hundred citizens.

A chain of four boxes: World, then `postcard::to_allocvec`, then FNV-1a, then a u64 of 8 bytes. Notes: anything in the world is in the checksum by construction, with no list of fields to forget to update; it is the same encoding a late joiner receives, so if the checksums agree the snapshot will too; and it is tagged with `checked_tick`, because a turn for `T` is sent three ticks early and the sender cannot yet know `T-1`. On the right, per tick on the wire: a player id, that tick's commands, usually none, and eight bytes of checksum, then the host's bundle of the same. Never on the wire: world state, positions, paths -- flow fields are two vectors of sixteen thousand cells each, so Nav lives outside World, rebuilt identically on every peer, in nobody's snapshot and nobody's checksum. Once, on late join: a Welcome of seed, tick and snapshot, 50 to 150 kB of postcard at five hundred citizens.

Slide 45

When someone lags, disagrees, or leaves

- The pipeline means a peer's reported tick is always behind the host's: a round trip old, and three ticks besides. A steady gap is health. A growing one is a peer falling behind, and one that stops moving is a peer that has stopped.

a peer falling behind

healthy

peers at [74, 71] — the host's own tick beside everyone's last reported one. A steady gap is the pipeline, not a fault.

5 seconds

shown as "waiting on ...". A backgrounded tab still sends turns, at 1 Hz, so this is the banner you actually see.

30 seconds

the host drops them and the game continues without that city.

refused, or stopped

build mismatch

Hello carries the wasm tab cannot even find the room, and is refused if it does.

desync

checksums for the same game stops with "desync with Alice at tick T".

the host leaves

over, for now. Because the sim is identical everywhere, migration is a reconnect rather than a state transfer.

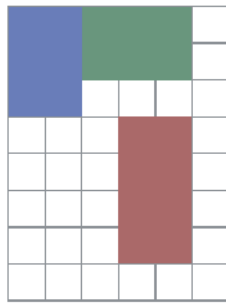
Known and unfixed: a checksum rides on Turn, which goes only to the host, so on the host can notice a desync — and on one, the joiner simply freezes without being told.

On the left, a peer falling behind in three states. Healthy: peers at 74 and 71, the host's own tick beside everyone's last reported one, where a steady gap is the pipeline rather than a fault. At 5 seconds it shows as 'waiting on', the banner you actually see, because a backgrounded tab still sends turns at 1 Hz. At 30 seconds the host drops them and the game continues without that city. On the right, refused or stopped. Build mismatch: Hello carries the wasm's sha256, so a stale tab cannot even find the room, and is refused if it does. Desync: checksums for the same tick differ and the game stops with 'desync with Alice at tick T'. The host leaves: over, for now, and because the sim is identical everywhere, migration is a reconnect rather than a state transfer. In red at the foot: known and unfixed -- a checksum rides on Turn, which goes only to the host, so only the host can notice a desync, and on one the joiner simply freezes without being told.

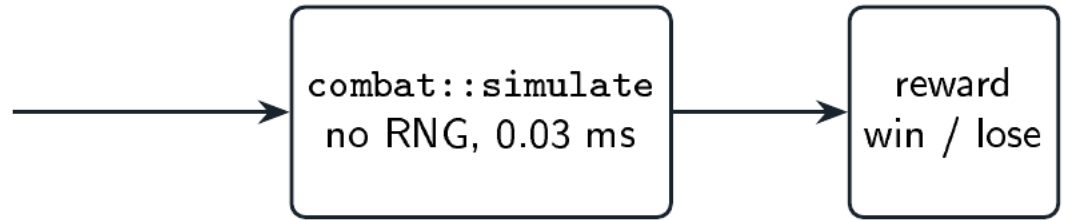
Slide 46

Gear Master

- Q-Learning over State Space
- Fights are simulations, no RNG, 0.03 ms per fight
- So you can evaluate millions of boards quickly
- The real decisions are how to shop and pack gear effectively (quartermaster), and how to reach every event in the game (Pathfinder)
- Two agents, and they don't share verbs



a board
(five of these)



Quartermaster (DQN)
Place Rotate Buy Sell
Reroll
scores (board, move) pairs

disjoint verb sets:
neither is offered
the other's move

Pathfinder
Fight
packin

A board of placed pieces feeds `combat::simulate`, no RNG at 0.03 ms, which produces a win or lose reward. A note in green reads: not an environment the agent is inside -- an exact objective, millions of evaluations. The board is captioned 'a board (five of these)'. Two agents below. Quartermaster, a DQN, has the verbs Place, Rotate, Buy, Sell and Reroll, and scores board-and-move pairs. Pathfinder, Q-learning, has the verbs Fight, Walk, Town and Answer, where packing is one macro-action. Between them: disjoint verb sets, neither is offered the other's move.

Slide 47

Gear Master

- How I started the Q-Learning work
- B1 Establish context
- B2 Require step-by-step justification
- B3 Build in self-questioning

the brief (mine)
six facts about the engine,
stated and marked "correct me
from the code"; no Python; a
search baseline must be beaten
before any net

rl-research.md (agent)
crates: burn, candle, tch-rs, rurel
...
prior art: Laterre 2018 ranked re-
ward; 2025 bin-packing review;
Crescitelli 2023; sts-light-speed;
MTG draft ranking
dated, (*unverified*) where it could
not read a date

B1

context: the engine's own
numbers, cited by line

B2

every recommendation names the
paper or the crate release behind
it

Three documents in sequence. The brief, mine: six facts about the engine, stated and marked 'correct me from the code'; no Python; a search baseline must be beaten before any net. rl-research.md, the agent's: the crates burn, candle, tch-rs and rurel; prior art including Laterre 2018 ranked reward, a 2025 bin-packing review, Crescitelli 2023, sts-lightspeed and MTG draft ranking; dated, and marked unverified where it could not read a date. rl-agent-plan.md, the agent's: 'right on five and a half', the purse is an input to combat -- one correction the brief could not know. Tagged B1, context is the engine's own numbers cited by line; B2, every recommendation names the paper or the crate release behind it; B3, the brief's own claims were the assumptions to check.

Slide 48

Gear Master

- Q-Learning Implementation deviated from design docs
- A2 Extract the rule the chatbot actually used
- A3 Locate the first divergence
- C4 Verify against hand-computed cases

	the design doc	what got built
state	11,040 floats 5 grids × 96 cells × 23	30 floats for the whole board
action	factored piece, then anchor + rotation	one shared 32-float move vector
agents	two disjoint verb sets	one encoder for both
road verbs	Fight Walk Town Answer	all of them _ =>

qmoves: 1,341 road verbs offered, 1 distinct vector the net can see

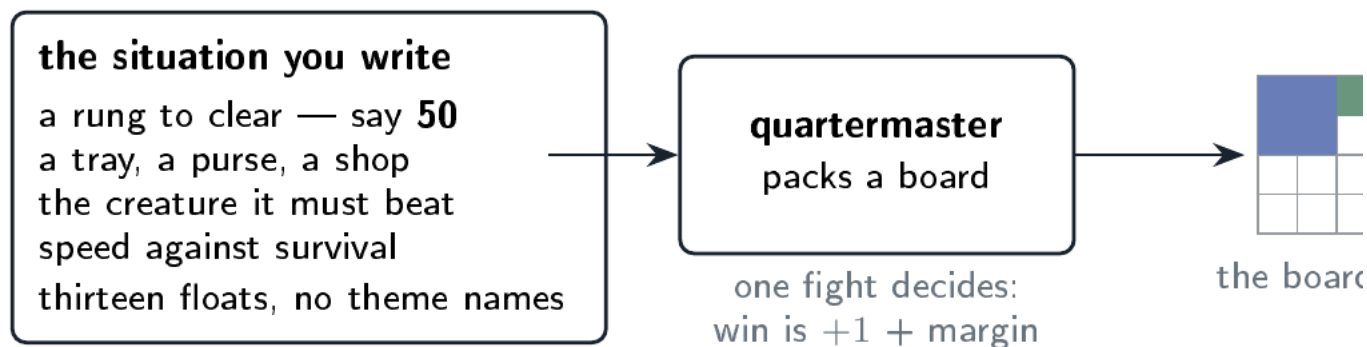
A two-column comparison of the design doc against what got built. State: 11,040 floats, five grids times 96 cells times 23, against 30 floats for the whole board. Action: factored, piece then anchor plus rotation, against one shared 32-float move vector. Agents: two, with disjoint verb sets, against one encoder for both. Road verbs: Fight, Walk, Town and Answer, against all of them collapsing to 8. A boxed line at the foot: qmoves -- 1,341 road verbs offered, 1 distinct vector the net can see.

Gear Master

- Pathfinding agent as validity solver for quest chains

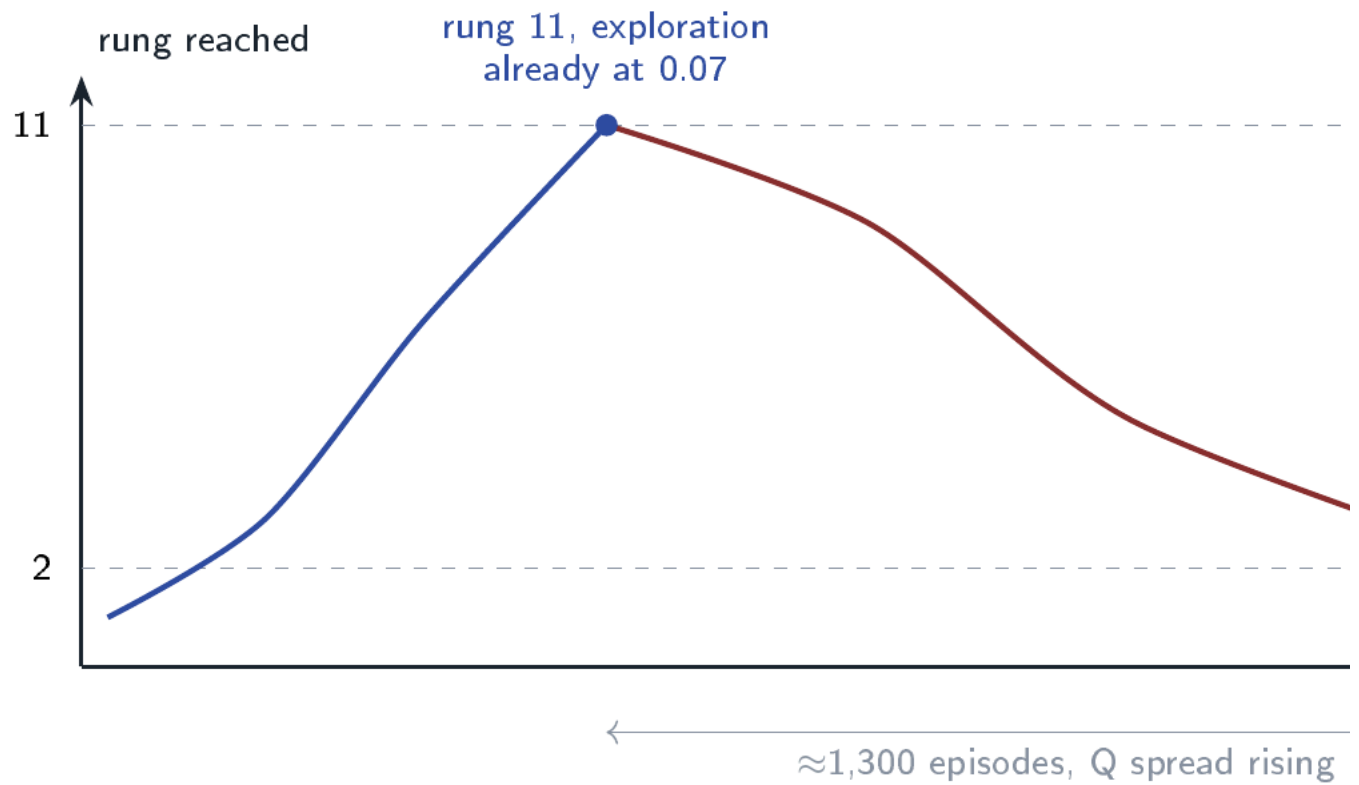
Gear Master

- Packer agent as enemy gear author



Bug

- Q-Learning overfitting
- A0 Calibrate your reliance
- A3 Locate the first divergence
- The packing agent got to rung 11
- Then slid back to rung 2 over 1,300 episodes
- The shaping reward was too quiet to hear
- And the saved nets were loading as None



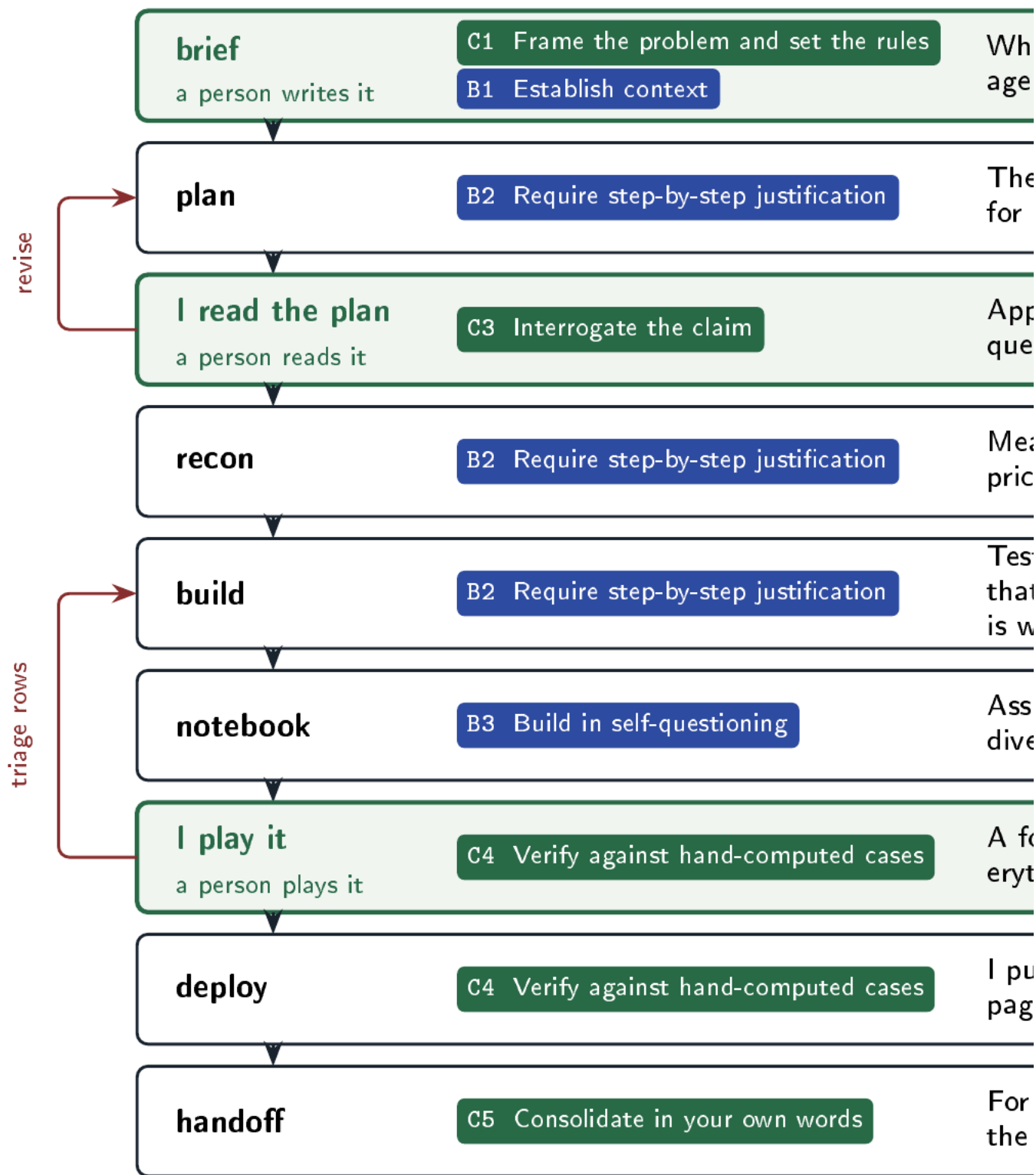
A curve of rung reached against episodes. It climbs to a marked peak at rung 11, annotated 'exploration already at 0.07', then falls away steadily to a marked point at rung 2. The decline spans about 1,300 episodes and is labelled 'Q spread rising'.

Slide 52

Question the system

- C6 Question the system
- Who built it, and on whose work?
- Whose math is it good at?
- Did the confident tone mislead you?
- What would you want to know before leaning on it?

Slide 53



A nine-stage vertical pipeline, each stage tagged with the procedure steps it serves. Brief, written by a person, tagged C1 and B1: what we are making, what done means, and what the agent may not decide on its own. Plan, B2: the agent fills the brief in, every decision and the reason for it, before any code. 'I read the plan', read by a person, C3: approve it or send it back, and a reference it cannot find is a question for me rather than a guess; a revise arrow loops back to plan. Recon, B2: measure against the running build, counts, brackets and prices, and say in the commit what you found. Build, B2: tests and content lints in the same change, and a number that disagrees with the plan must name which of the two is wrong. Notebook, B3: assumptions listed and checked while the work happens, as a divergence, a finding, or a decision handed back. 'I play it', played by a person, C4: a fortnight of real play against what the plan claimed, with everything wrong becoming a triage row, looping back to build. Deploy, C4: I push, and drive.py then walks every check against the live page in three engines. Handoff, C5: for whoever is next, the rules that are load-bearing and the mistakes that cost a day.

Resources

- github.com/sgilson7/floodline
- WebRTC lockstep, no server
- github.com/sgilson7/gear-master
- engine, agents, the RL write-ups
- github.com/sgilson7/gear-master-2d
- the full doc set, tests, gate
- github.com/sgilson7/pdf-redactor
- github.com/sgilson7/perturbation-workbench

Questions?

- Sam Gilson
- sagilson@ncsu.edu

Where this sits

The three procedures on these slides have a page each, and they are in the concept graph with the terms from the papers they came out of. The publications list this talk with its date and course.